A Single-Board Fermilab Timing Pipeline on the Xilinx KR260: Decode, Timestamp, Publish, Mirror

Jacob Rossel[1,2]*, Evan Milton[2]

[1]Department of Electrical Engineering and Computer Sciences, University of California, Berkeley
[2]Fermi National Accelerator Laboratory, Batavia, IL

Abstract

Fermilab's accelerator timing links broadcast short event codes to thousands of devices at once, but the links themselves carry no absolute notion of time; that comes separately from a White Rabbit reference. This work builds the piece that ties the two together on a single board. On a Xilinx Kria KR260 (Zynq UltraScale+), the programmable logic decodes a real Fermilab TCLK link, stamps every event with an absolute White-Rabbit {sec, ns} UTC time, and reads the timestamped stream out over AXI4-Lite; a thin Linux process on the same die publishes each event into a Redis stream on the control network. To exercise the full chain on one board, the decoded events are re-encoded as gigabit ACLK, transmitted out an SFP+ optical port, looped back over a short fiber jumper, and decoded again on the same timeline, and are additionally mirrored as an ACLK-Lite Manchester waveform for benchtop probing. Across sustained, multi-day testing against real Fermilab TCLK, the pipeline has decoded, timestamped, and published many events with practically zero loss, and folding the timestamped stream on the 60-second accelerator supercycle recovers the machine's periodic structure directly from the published data.

1 Introduction and motivation

A particle accelerator is a distributed machine whose thousands of devices, from magnet power supplies to beam instrumentation to the injection kickers, all have to act in step. They are kept in step by a family of *timing links*: serial broadcasts that carry short event codes ("beam is coming", "supercycle starts now", "reset this counter") to every device at once. At Fermilab three of these links are relevant to this project, and they are three generations of the same idea [1]:

- **TCLK** is the legacy link: a roughly 10 MHz biphase-mark (Manchester) serial stream on a 3.3 V copper line. Each frame carries an 8-bit event code, written $XX in Fermilab notation, and nothing else. There is no timestamp on the wire: a device knows *what* happened and *that it just happened*, but the link itself does not say *when* in any absolute sense.
- **ACLK** is the newer gigabit link. It carries the same style of event, but as an 8b/10b framed packet on a 1.25 Gbps optical (SFP) transport, with room for a 16-bit event field, a 64-bit data payload, and a CRC-8. It runs far faster than TCLK, and the CRC-8 lets a receiver reject a corrupted frame.
- **ACLK-Lite** is the PIP-II down-converted variant [2]: a Manchester stream meant to be carried on a simpler physical layer for the new linac, closer in spirit to TCLK than to full gigabit ACLK.

None of these links carries an absolute, disciplined notion of time on its own. That comes separately from **White Rabbit (WR)** [3, 4], a precise timing distribution standard that delivers a 10 MHz reference plus a one-pulse-per-second (PPS) tick. WR is the source of *when*; the timing links are the source of *what*.

The goal of this project is to build the piece that ties *what* to *when* and gets the result to software. Concretely: receive real Fermilab timing events on a single board, put an absolute {sec, ns} UTC timestamp on each event, deliver the timestamped stream to software over a standard interface, and, so that the whole thing can be exercised end to end without a second board, also re-broadcast the events back out over the accelerator's own timing-link transports. The deliverable is one FPGA bitstream plus the board-side software that reads it out.

2 System overview

The target is a **Xilinx Kria KR260** (Zynq UltraScale+, part `xck26-sfvc784-2LV-c`) [5], a system-on-module

*Corresponding author: jacob_rossel@berkeley.edu. Work performed during a 2026 summer internship at Fermilab under the U.S. DOE SULI program.

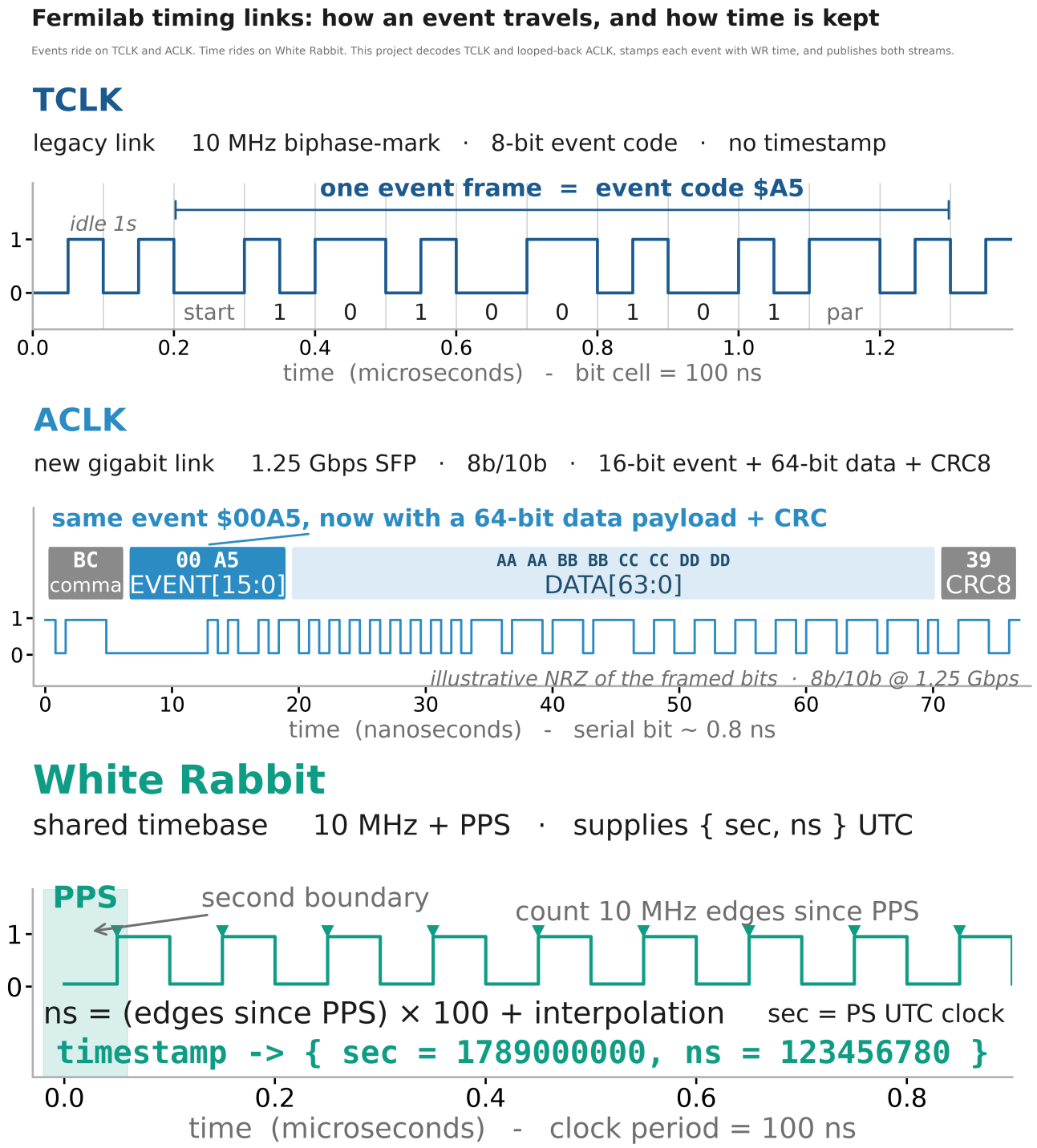

Figure 1: *The three links. TCLK carries an 8-bit event code as a 10 MHz Manchester stream with no timestamp. ACLK carries the same event, now framed in 8b/10b at 1.25 Gbps with a 64-bit payload and a CRC-8. White Rabbit supplies the shared timebase: nanoseconds come from counting 10 MHz edges since the last PPS and interpolating, seconds come from the UTC clock, and the result is stamped onto every event as {sec, ns}.*

that pairs a hardened ARM processing system (PS) running Linux with a large block of FPGA programmable logic (PL) on the same die, joined by AXI interconnect. The design is built around that split: the PL does everything that has to be fast and deterministic (recover bits from a wire, frame them, latch a hardware timestamp the instant an event arrives), and the PS does everything that benefits from a general-purpose computer (poll the hardware, format records, talk to the network).

What makes this a single-board loop rather than just a receiver is that the board also plays back what it receives. Real TCLK comes in on a copper Pmod pin. The board decodes it, stamps it, and reads it out as one event source. In parallel it re-encodes those same events into ACLK frames and transmits them out the SFP+ optical port through the FPGA's gigabit transceiver. A short fiber jumper loops that transmission straight back into the board's receiver, where it is decoded again, stamped again against the *same* White Rabbit timeline, and read out as a second, independent event source. Because both readouts share one timebase, a TCLK event and its looped-back ACLK twin carry directly comparable absolute times, which is exactly what you need to validate that the encode/transmit/receive/decode round trip is faithful. Finally the decoded ACLK events are mirrored a third time as an ACLK-Lite Manchester waveform on a Pmod output pin, available as a scope probe.

3 Physical I/O

The KR260 carrier card exposes its PL user I/O through four 12-pin Pmod connectors and a Raspberry Pi header. These interfaces all use 3.3 V LVCMOS33 signaling and are connected to the FPGA through auto-direction level translators, so any pin can serve as an FPGA input or output. All of the low-speed link signals in this design land on **PMOD1** (Figure 3).

TCLK does not need a clock-capable FPGA pin. The decoder oversamples the line at 80 MHz and treats it as ordinary data (Section 4.1), so any general-purpose LVCMOS33 user pin works; a 10 MHz biphase-mark stream has edges no closer than about 50 ns apart, comfortably inside the translator's bandwidth. The gigabit ACLK link is the exception: it does not use a Pmod pin at all but the dedicated SFP+ cage, driven by the FPGA's gigabit transceiver (GTH) differential pairs and a 156.25 MHz reference clock [6].

4 The signal chain in technical depth

4.1 TCLK decode

Biphase-mark coding (the "mark" form of Manchester) puts a transition at the start of every bit cell and adds a second transition in the middle of the cell only for a one. The clock is therefore embedded in the data, which tolerates noise and clock drift, but it means the receiver has to recover bit boundaries from edge timing rather than from a separate clock line. The design does this by brute-force oversampling: `serdec4_9MHz` samples the raw line at 80 MHz (eight samples per 100 ns bit cell) into a shift register and recovers the biphase-mark cells from the sampled edge pattern. Downstream, `TCLK_DESERIALIZER2` and `TCLK_RCV` assemble the recovered cells into frames and pull out the 8-bit event code. The whole TCLK front end plus its readout and timestamp lives in `tclk_readout_top`.

The sampler runs faster than the line rate instead of at it because the bit boundaries land at an arbitrary phase relative to the FPGA's own clock. An oversampled, edge-detected recovery tolerates that phase far better than a sampler clocked at the nominal line frequency would.

4.2 The White Rabbit timebase

`wr_timebase` turns the White Rabbit 10 MHz reference and PPS into a free-running {sec, ns} counter. The nanosecond field is built by counting 10 MHz edges since the most recent PPS edge and multiplying by 100 ns per edge, with local-clock interpolation between edges for finer resolution,

$$t_{\mathrm{ns}} = 100\,\mathrm{ns} \times N_{10\,\mathrm{MHz}} + \delta_{\mathrm{interp}}, \tag{1}$$

while the seconds field comes from the PS UTC clock latched at the second boundary. The PPS edge resets the nanosecond count every second, which keeps it disciplined to the White Rabbit master rather than free-running off the FPGA oscillator.

The block enforces **strict validity**. If an event is stamped while the board is not yet White-Rabbit-synchronized, the timestamp is emitted as zero rather than as a plausible-looking but meaningless number, and software surfaces that to the operator as `UNSYNC`. This is a safety property: a wrong timestamp that looks right is worse than an obviously invalid one, because a physicist correlating events after the fact has no way to tell a subtly wrong time from a correct one.

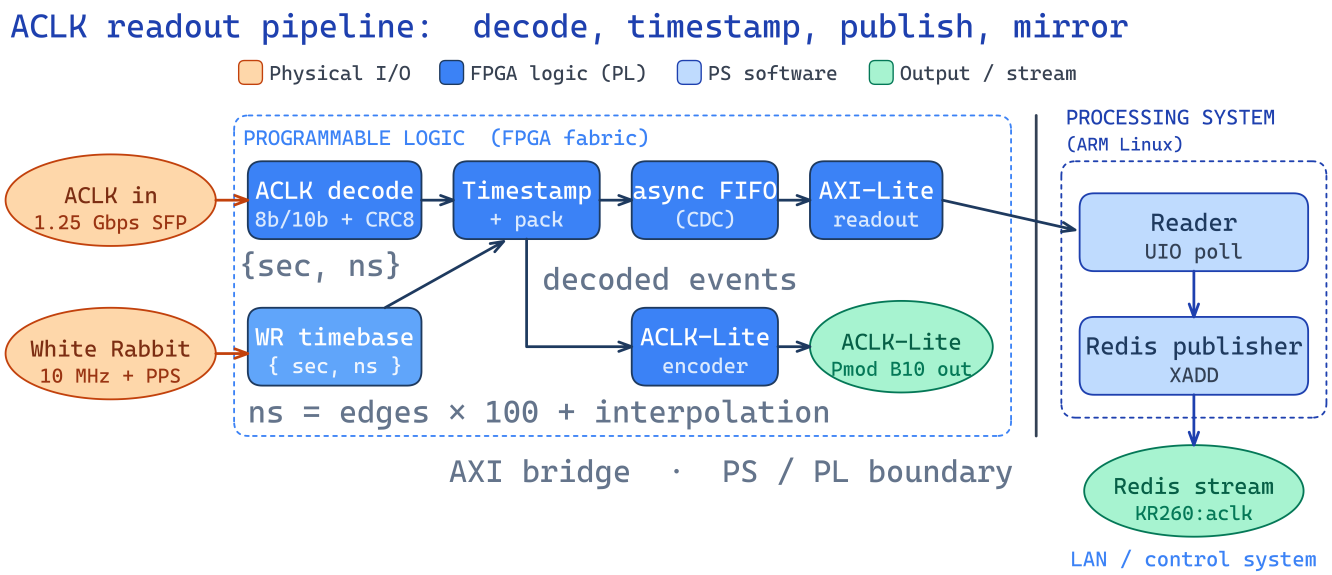

Figure 2: *The ACLK receive, timestamp, readout, and publication pipeline. In the PL, the design decodes the link, latches a White-Rabbit timestamp, buffers the result across clock domains in an asynchronous FIFO, and exposes it over AXI4-Lite. In the PS, a reader polls the hardware and a publisher pushes each event into a Redis stream on the control network. The ACLK-Lite encoder mirrors the decoded event onto a Pmod pin as a scope probe.*

3.3V	GND	7	5	3	1
3.3V	GND	8	6	4	2

Figure 3: *The Pmod connector layout. On PMOD1: pin 1 (package H12) is the TCLK biphase-mark input; pin 2 (B10) is the ACLK-Lite Manchester mirror output; pin 3 (E10) is the White Rabbit 10 MHz reference; pin 4 (E12) is the White Rabbit PPS. The two leftmost columns are the fixed 3.3 V and ground rails.*

Both event readouts are stamped from `wr_timebase`, instantiated as two replicas that share the same reference so the TCLK and ACLK paths sit on one timeline. A third register slave, `wr_timebase_axi`, exposes the timebase to software for monitoring and arming.

4.3 Sub-sample edge timing

The event timestamp is latched on the 200 MHz stamp clock, so a stamp resolves to one 5 ns tick. A separate block measures where inside that tick the event's reference edge actually fell. The clocking wizard produces four copies of the 200 MHz clock at 0, 90, 180, and 270 degrees, and the incoming edge is sampled against all four; which pair of phases the transition falls between identifies one quarter of the tick. Ideally those quarters are 1.25 ns wide. That is the ideal width and not a measured one: the real bin boundaries follow the MMCM's actual phase relationships, which sit close to but not exactly 90 degrees apart, so anything that cares about the absolute sub-tick number should calibrate the bins on its own board (`deploy/fine_calibrate.py` does this from the event density of a capture). The result is packed into two spare bits of the event's FLAGS word next to a validity bit, so software that ignores those bits sees exactly the record it saw before.

The block adds to the decode path without disturbing it. It taps a frame-detection strobe out of the deserializer and never gates or reclocks the decode logic, and every event carries its own `fine_valid` bit, so failing to resolve the sub-tick degrades that event to the plain 5 ns stamp instead of losing it. On hardware the fine bits resolve on essentially every event, including on the real 3.3 V line, which was the open question: ringing on a real copper line was the plausible reason for the sub-tick decode to fail there, and no simulation could settle it.

The extra resolution buys little today. The event-to-event spread seen on TCLK is dominated by something upstream of the board and far coarser than either 5 ns or 1.25 ns: repeated arrivals of the same event code do not scatter smoothly but land on a grid a couple of hundred nanoseconds wide. Nothing on the receiving end can improve that, and the board's own contribution to the spread is already small next to it. So the fine bits

work, and they cost almost nothing, but on TCLK they measure well below the term that actually dominates.

The case for keeping them is the ACLK path. Once events arrive over ACLK on the SFP instead of over TCLK on copper, the arrival edge should be considerably more repeatable: a 1.25 Gbps serial frame pins its own arrival more than a hundred times more finely than a 100 ns TCLK bit cell does, and an optical receiver does not have the ringing and reflection behavior that a 3.3 V copper input has. The caveat is that this work could not separate how much of the observed spread comes from the link itself and how much comes from when the event was issued upstream, and only the first part improves by changing the transport. That expectation has been reasoned through and not tested.

4.4 The readout core and AXI4-Lite interface

Both the TCLK and the ACLK readouts are built from the same two modules, which is what keeps the two paths behaving identically. `aclk_readout_core` latches a 64-bit hardware timestamp the instant an event is presented, packs the event through a null-drop stage (events that decode to a null code are discarded in hardware so they never reach software), and carries the packed record across the clock-domain boundary through a dual-clock `async_fifo`. Crossing that boundary safely matters because the decode logic and the AXI interface run on different clocks; the async FIFO with Gray-coded pointers is the standard, verified way to hand data between them without metastability.

`aclk_readout_axi` wraps that core in an AXI4-Lite [7] register block that the PS reads. Software sees a small register file: status (FIFO empty / overflow), the event code and its flags, the 64-bit data payload split into high and low words, the 64-bit timestamp split into high and low words, a write-only POP register to advance the FIFO, running event / null / error counts, health registers (line activity, receive-clock heartbeat, MMCM and WR lock), and a 256-bit event drop-mask filter (`FILTER_CFG` to program which event codes to discard in hardware, `FILTERED_COUNT` to report how many were dropped).

One hardware-specific quirk shaped the register layout: on the KR260's low-power-domain AXI path, the hand-written AXI4-Lite slave only returns read data correctly at 16-byte-aligned offsets, so every register is spaced 16 bytes apart rather than packed at 4-byte word boundaries. The full field-level register map for both readouts and the WR monitor slave is in the project's generated hardware interface guide; this report gives the shape rather than transcribing every bit.

4.5 TCLK to ACLK re-encode and the gigabit loop

To close the loop, decoded TCLK events are re-encoded as ACLK frames. `aclk_tclk_encoder` gearboxes each 8-bit TCLK event into the wider ACLK frame format (event field, data payload, and a CRC-8 computed by `crc8_calc`), using the 16-to-96 and 96-to-16 bit gearboxes that match the transceiver's internal width to the frame width. The framed stream is 8b/10b encoded and driven out the SFP+ by `aclkgt_gt`, the Xilinx gigabit transceiver (GTH) wizard IP [6] configured for 1.25 Gbps with a 156.25 MHz reference clock, committed to the repository as a generated `.xci`.

An external fiber jumper loops the transmitter's optical output straight back into the same transceiver's receiver. On the receive side `ACLK_RCV` and the matching gearboxes and CRC-check logic decode the frame back into events, which `aclk_gt_readout_top` stamps against the shared White Rabbit timeline and reads out through the second `aclk_readout_axi` instance. This is the mechanism that lets one board exercise the full encode / serialize / optical-link / deserialize / decode chain: the ACLK source in the data is literally the board's own TCLK events after a complete round trip through the gigabit link.

4.6 The ACLK-Lite mirror

The last stage takes the decoded ACLK events and mirrors them out as an ACLK-Lite Manchester waveform. `aclk_lite_bridge` adapts the decoded event interface to the encoder, and `aclk_lite_encoder` drives the biphase-mark output on PMOD1 pin 2 (B10). This output is a scope probe: it makes the down-converted PIP-II-style link observable on a benchtop oscilloscope and provides a physical ACLK-Lite source for testing downstream receivers, all without any additional hardware. The exact on-wire framing for ACLK-Lite (per-byte parity, byte-oriented cells, the terminal-cell convention, and the 1/2/12-byte frame lengths) is documented authoritatively in the ACLK-Lite interface specification [8].

5 Software readout path

On the PS side the design is deliberately thin, because the hard real-time work has already been done in the PL. Two small Python readers, one per source, map

the AXI4-Lite register block through Linux UIO and poll it: read the status register, and while the FIFO is non-empty, read the event, its flags, its data payload, and its 64-bit timestamp, then write the POP register to advance to the next record. Reading through UIO rather than a custom kernel driver keeps the board-side software simple and portable.

A reader hands its events to a Redis publisher, which pushes them into Redis streams [9] on the control network with `XADD`. The key schema is the laboratory's rather than this project's: a braced base key, `{TCLK}`, with the braces present so that a Redis Cluster hashes the whole set of keys to one slot, and three streams written per event (that occurrence's time under its own event code, a running per-code occurrence counter, and one combined feed carrying the code itself). Each entry's stream ID comes from the event's White Rabbit time rather than from Redis's own arrival time, with a monotonic guard that bumps a duplicate or backward time to the previous ID plus one nanosecond instead of letting `XADD` fail. Persistence is off, because this is live telemetry and not a database of record; a separate on-disk archive keeps the durable copy. A consumer anywhere on the network then sees timestamped accelerator events in event-time order, sliceable by event code. The explicit stream IDs work on Redis 6 and later, though the publisher's watchdog uses hash-field expiry and wants 7.4.

Only the TCLK source is wired to the publisher today. ACLK is decoded, timestamped on the same timeline, and available at its own readout, but it is not published. Recent effort went into landing the TCLK stream on the key space the laboratory's downstream consumers were already waiting for. Publishing ACLK as well is mostly configuration: the publisher is not specific to a source, so it needs a base key for ACLK and the second reader pointed at it.

6 Operating notes: the White Rabbit reference

Getting the White Rabbit reference into the board took work on the bench and work in the operating procedure, and none of it shows up in the RTL. This section records it.

6.1 Getting the two signals into the board

The White Rabbit node's 10 MHz output is a sine centered on zero volts, so it swings negative, and a negative voltage on a Pmod pin can damage the board. It cannot be connected directly. A series capacitor removes the negative excursion, and a pair of resistors, one to ground and one to the 3.3 V rail, biases what is left so the waveform crosses the input's switching threshold on every cycle. With that in place the PL sees a usable 10 MHz square wave and the timebase's cells-per-second count sits where it should.

The PPS needed a different fix. The source expects a 50 Ω termination and the Pmod input is high impedance, so the pulse reflected off the unterminated end of the coaxial cable and arrived back at the input as extra edges. A 180 Ω resistor from the cable's center conductor to ground at the board end damps the reflection while still leaving enough amplitude at the pin to cross the logic threshold.

Both are hand-built on the bench and the timebase does not work without them. Anyone restoring the setup after the board has been moved or powered off should check both before suspecting the firmware.

6.2 Spurious PPS edges

The timebase counts PPS edges, so anything the PL mistakes for a PPS edge adds a whole second to every stamp that follows. That is not a subtle error and it does not correct itself. The PL therefore filters the PPS line and counts what it discards in a `PPS_REJECT` register. That count moved during bench work, and every rejected edge is a second the clock would otherwise have gained. Read `PPS_REJECT` at the start and the end of every capture. If it changed, something disturbed the PPS line during the run and that run's absolute times should be treated with suspicion; re-arm before starting the next one.

6.3 Frequency offset against GPS

The stamped seconds are PPS periods by construction, so if the reference's frequency does not match GPS exactly, the board's notion of time walks away from real time at that rate. The offset was measured several times over the course of testing and it was not the same number each time. This work did not run long enough, or often enough, to say whether it drifts slowly, steps, or settles. Any single measurement should therefore be treated as belonging to its own run rather than to the installation, and a constant should never be baked into an analysis. Every capture contains what is needed to calibrate itself: `deploy/gps_calibrate.py` measures the offset from the GPS-locked marker events in the capture's own data. Characterizing how the offset behaves over longer spans is a self-contained job for

whoever picks the project up, and the board already has everything needed to do it.

6.4 What an interruption costs

The timebase is strict. An interruption of either the PPS or the 10 MHz unlocks it permanently and latches a sticky lost-lock flag, and every event stamped afterward is marked UNSYNC. The publisher drops UNSYNC events rather than publishing them, so an interrupted run carries gaps instead of plausible wrong times, which is what the strict-validity rule described earlier is for. Recovery is a re-arm, and the capture launcher runs a guard process that re-arms automatically after a blip, so an interruption costs seconds instead of the run. Check the lock health reported in the capture statistics before trusting a run's data.

7 Verification and results

Development followed a simulation-first discipline. The inner loop is a cocotb (Python) testbench suite [10] run under Icarus Verilog [11], with one testbench per module and an end-to-end chain testbench (`aclk_pipeline_chain`) that exercises the whole pipeline in simulation before any bitstream is built. Every stage (TCLK receive, ACLK receive, the readout and its AXI register map, the async FIFO, the encoders, the White Rabbit timebase) has its own testbench, and each testbench produces a diagnostic Matplotlib plot. The board-side Python has its own `pytest` suite covering the register map, the Redis publisher, and the statistics reconciliation.

The primary hardware validation is extended continuous operation against real Fermilab TCLK. Across that testing the board decoded real TCLK, looped every event through the ACLK gigabit fiber path, stamped both sources against the shared White Rabbit timeline, and published the TCLK stream into Redis, accumulating many events with practically zero loss (verified by reconciling the hardware event counters against the number of records that reached Redis and the on-disk statistics log). This exercises TCLK decode, the White Rabbit arm and lock, the gigabit SFP loop, and both readouts together under sustained real load. The hardware event drop-mask ran over the same capture, discarding a selected event code in the PL before it reached software.

Because every event carries an absolute timestamp, the captured stream can be analyzed the way a physicist would analyze real accelerator data. Folding every event on the 60-second accelerator supercycle reconstructs the machine's timing schedule directly from the published Redis stream (Figure 4). Isolating a single event code then makes the per-cycle detail visible, both its offset distribution and its cycle-to-cycle stability (Figure 5).

8 Discussion and conclusion

A single Xilinx KR260 now closes the whole timing-link loop. It receives a real Fermilab TCLK broadcast, puts an absolute White-Rabbit {sec, ns} timestamp on every event, re-encodes the stream as gigabit ACLK and loops it back over fiber to produce a second, independently decoded source on the same timeline, mirrors the result as ACLK-Lite for a scope, and publishes the timestamped TCLK stream into Redis for any consumer on the control network. Extended testing against real TCLK, with many events published at practically zero loss, shows that the pipeline holds up under sustained real load, and recovering the 60-second supercycle straight from the published timestamps shows that the timestamps are physically meaningful, not merely present.

The design choices that mattered most protect correctness rather than add features: stamping both sources from replicas of one White Rabbit timebase so the two sources are directly comparable, emitting an explicit invalid timestamp instead of a plausible wrong one when the board is not yet synchronized, and crossing every clock-domain boundary through a verified Gray-coded async FIFO. The single-board re-encode-and-loop-back structure is what makes the system testable without a second board or a live upstream feed: the ACLK source in the data is the board's own TCLK after a full round trip through the gigabit link.

One validation item remains on the ACLK path itself. It is today exercised only through the board's own re-encode and fiber loopback, not against a live upstream accelerator ACLK feed, so confirming the ACLK CRC-8 polynomial against a real ACLK source remains the main deferred item. It is a concrete next step rather than an open design question, and it does not block the single-board pipeline that this work delivers. That real ACLK source also points to the eventual deployment. Once an upstream ACLK feed exists, as it will on PIP-II, the same receiver can take that ACLK straight into the transceiver and bypass the TCLK decode and re-encode entirely, so the board timestamps and publishes real PIP-II events directly from their own ACLK link rather

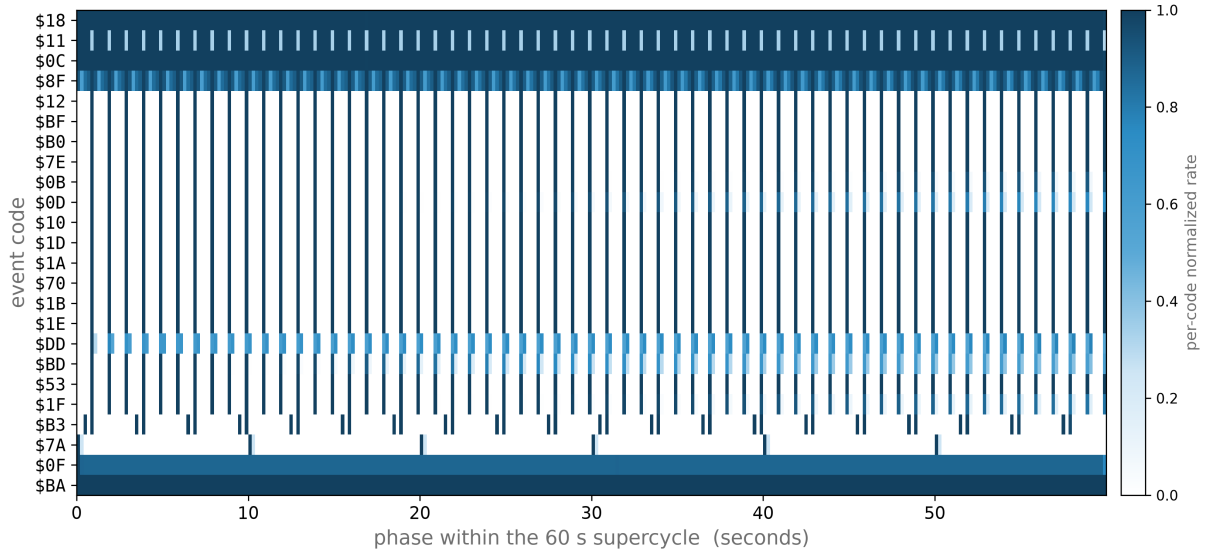

Figure 4: *The machine's timing schedule, reconstructed purely from the White-Rabbit timestamps in the captured stream. Every event from 340 clean 60 s supercycles is folded onto one supercycle (anchored on the $00 reset code); the top 30 event codes are shown as rows, phase within the supercycle runs left to right, and each row is normalized to its own peak, so color shows* when *a code fires within the cycle rather than how often relative to other codes. The always-on 15 and 20 Hz reference combs (for example $BA, $0C, $0F) fill their rows, while scheduled events land at sharp, repeatable phases, many of them aligned to common machine-cycle boundaries.*

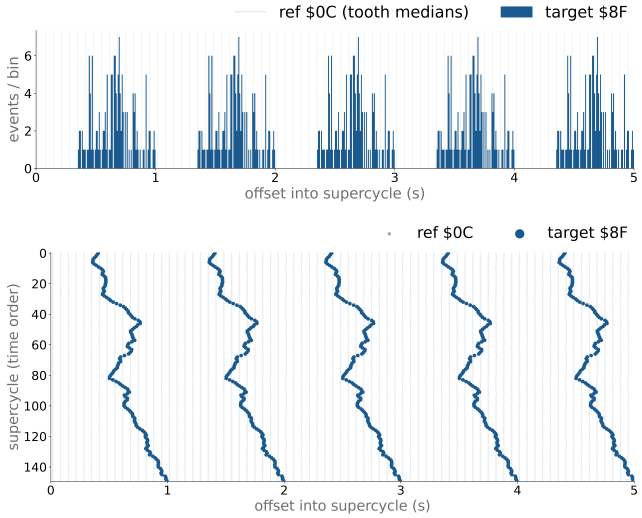

Figure 5: *Zooming into a single event code ($8F) over the first 5 s of the 60 s supercycle, folded across 150 supercycles anchored on the $00 reset (the faint vertical comb is the $0C reference). Top: the folded-offset histogram of $8F, its distribution shape within the window. Bottom: the same events as a raster, one row per supercycle in time order. The stable, gently drifting bands confirm that $8F lands at consistent phases across the whole capture, which is the timestamp chain doing its job end to end.*

than TCLK events routed through the loopback. The decode, White-Rabbit timestamp, and Redis-publish core stays exactly as it is; the re-encode-and-loopback path that makes single-board testing possible today simply falls away.

9 Where to pick this up

Four things are worth doing next, in roughly this order.

- Wire the ACLK reader into the Redis publisher. Those events are already decoded, stamped on the shared timeline, and waiting at their own readout; what is missing is a base key for the source and the second reader pointed at the publisher. This is the smallest of the four and the one a downstream consumer would notice.
- Confirm the ACLK CRC-8 against a live upstream ACLK feed. The fiber loopback proves the board agrees with itself, which is not the same as agreeing with the accelerator.
- Characterize the White Rabbit reference's frequency offset over time. Measure it per run with `gps_calibrate.py`, keep the results, and find out how the offset behaves over weeks instead of hours. That answer decides whether the published timestamps can be trusted in absolute terms or only as a self-consistent timeline.
- Re-evaluate the sub-sample edge timing of Section 4.3 once events arrive over ACLK instead

of copper TCLK, which is where that resolution would start to matter.

Day-to-day operation, cold-start recovery, and console access live in the repository's operations runbook and handoff notes.

Acknowledgments

The authors thank the Fermilab accelerator controls group for their support. This manuscript has been authored by FermiForward Discovery Group, LLC under Contract No. 89243024CSC000002 with the U.S. Department of Energy, Office of Science, Office of High Energy Physics. This work was supported in part by the U.S. Department of Energy, Office of Science, Office of Workforce Development for Teachers and Scientists (WDTS) under the Science Undergraduate Laboratory Internships Program (SULI). The design (RTL, simulations, and board-side software) is available in the project repository.[1]

References

[1] Fermi National Accelerator Laboratory. *Beam Synchronus for the Rest of Us*. Fermilab DocDB. Fermilab Beams-doc-10499-v5. 2024.

[2] Fermi National Accelerator Laboratory. *PIP-II Timing Interface Specification Document (ED0016516)*. Fermilab DocDB. Fermilab Beams-doc-10499-v5. 2024.

[3] J. Serrano, P. Alvarez, M. Cattin, et al. "The White Rabbit Project". In: *Proc. 12th Int. Conf. on Accelerator and Large Experimental Physics Control Systems (ICALEPCS)*. 2009.

[4] IEEE. *IEEE Standard for a Precision Clock Synchronization Protocol for Networked Measurement and Control Systems*. IEEE Std 1588-2008. 2008.

[5] AMD Xilinx. *Kria KR260 Robotics Starter Kit*. `https://www.amd.com/en/products/system-on-modules/kria/k26/kr260-robotics-starter-kit.html`.

[6] Xilinx. *UltraScale Architecture GTH Transceivers User Guide (UG576)*. 2021.

[7] ARM. *AMBA AXI and ACE Protocol Specification (IHI 0022)*. 2021.

[8] Fermi National Accelerator Laboratory. *ACLK-Lite Interface Specification*. Fermilab DocDB. Fermilab Beams-doc-10499-v5. 2024.

[9] Redis Ltd. *Redis Streams*. `https://redis.io/docs/latest/develop/data-types/streams/`.

[10] cocotb developers. *cocotb: A Coroutine-Based Cosimulation Testbench Environment*. `https://www.cocotb.org`.

[11] Stephen Williams. *Icarus Verilog*. `http://iverilog.icarus.com`.

[1] `https://github.com/jRossel0310/tclk-aclk-bridge`